UNIVERSIDAD AUTÓNOMA DE SINALOA

FACULTAD DE INFORMÁTICA CULIACÁN

MANUAL DE PRÁCTICAS DE MONGODB

COLECCIÓN, CONSULTA, ACTUALIZACIÓN Y ELIMINACIÓN DE DOCUMENTOS CON MONGODB



ASIGNATURA: Bases de Datos Emergentes

PROGRAMA EDUCATIVO: Licenciatura en Ingeniería en Ciencia de Datos

RESPONSABLE: Dra. Lidia Yadira Pérez Aguilar

FECHA: agosto 2026

DATOS • CONOCIMIENTO • DECISIONES

Tecnología que transforma información en valor

PRÁCTICA 4. Proyección, ordenamiento y paginación

Objetivo

Controlar los campos devueltos por una consulta, ordenar resultados de forma ascendente y descendente, limitar la cantidad de documentos mostrados y aplicar paginación mediante skip() y limit() sobre la colección productos.

Actividades

- Ejecutar el código en el orden indicado y verificar los resultados.
- Registrar evidencia de las consultas ejecutadas.
- Comparar consultas completas y consultas con proyección.
- Interpretar los resultados en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Respuestas a los ejercicios.
- Interpretación breve de los resultados obtenidos.

0. Preparar el entorno de trabajo

- **Poner en uso nuestra base de datos**

```
use ventas_online
```

- **Contar los documentos**

```
db.productos.countDocuments()
```

Resultado esperado: 50 documentos.

1. Proyección de campos

La proyección permite decidir qué campos queremos mostrar en los documentos recuperados

1.1 Mostrar solamente SKU, nombre y precio

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
)
```

Aquí 1 significa que el campo será incluido y _id:0 evita mostrar el identificador generado automáticamente por MongoDB.

1.2 Mostrar SKU, nombre, categoría y stock

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        categoria: 1,
        stock: 1
    }
)
```

1.3 Aplicar filtro y proyección al mismo tiempo

Buscar productos de la categoría `"Audio"` mostrando únicamente SKU, nombre y precio.

```
db.productos.find(
    { categoria: "Audio" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
)
```

1.4 Mostrar todos los campos excepto stock

```
db.productos.find(
    {},
    {
        stock: 0
    }
)
```

2. Ordenamiento de resultados

MongoDB utiliza `sort()` para ordenar documentos.

1 = ascendente
-1 = descendente

2.1 Ordenar productos por precio de menor a mayor

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({ precio: 1 })
```

2.2 Ordenar productos por precio de mayor a menor

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({ precio: -1 })
```

2.3 Ordenar productos por stock de menor a mayor

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        stock: 1
    }
).sort({ stock: 1 })
```

2.4 Ordenar alfabéticamente por nombre

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1
    }
).sort({ nombre: 1 })
```

3. Ordenamiento por varios campos

También podemos establecer más de un criterio.

3.1 Ordenar primero por categoría y después por precio

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        categoria: 1,
        precio: 1
    }
).sort({
    categoria: 1,
    precio: -1
})
```

Esto significa:

1. ordenar las categorías alfabéticamente;
2. dentro de cada categoría, ordenar los productos de mayor a menor precio.

3.2 Ordenar por categoría y posteriormente por stock

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        categoria: 1,
        stock: 1
    }
).sort({
    categoria: 1,
    stock: 1
})
```

4. Limitar resultados con limit()

`limit()` permite controlar cuántos documentos devuelve una consulta.

4.1 Mostrar solamente 5 productos

```
db.productos.find().limit(5)
```

4.2 Mostrar los 5 productos más caros

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    precio: -1
}).limit(5)
```

Este ejemplo introduce el concepto de Top-5**.**

4.3 Mostrar los 10 productos con menor stock

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        stock: 1
    }
).sort({
    stock: 1
}).limit(10)
```

Esto puede interpretarse comercialmente como una forma inicial de identificar productos con existencias bajas.

5. Top-N

Podemos combinar filtro, proyección, ordenamiento y límite.

5.1 Mostrar los 5 productos más caros de Computo

```
db.productos.find(
    { categoria: "Computo" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    precio: -1
}).limit(5)
```

5.2 Mostrar los 3 productos de Audio con mayor stock

```
db.productos.find(
    { categoria: "Audio" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        stock: 1
    }
).sort({
    stock: -1
}).limit(3)
```

5.3 Mostrar los 5 accesorios más económicos

```
db.productos.find(
    { categoria: "Accesorios" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    precio: 1
}).limit(5)
```

6. Paginación con `skip()` y `limit()`

La paginación permite dividir un conjunto grande de resultados en grupos más pequeños.

Con 50 documentos podemos trabajar con páginas de 10 productos.

6.1 Primera página

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
```

```
        nombre: 1,
        precio: 1
    }
).sort({
    sku: 1
}).skip(0).limit(10)
```

Productos 1–10.

6.2 Segunda página

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    sku: 1
}).skip(10).limit(10)
```

Productos 11–20.

6.3 Tercera página

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    sku: 1
}).skip(20).limit(10)
```

Productos 21–30.

6.4 Cuarta página

```
db.productos.find(
    {},
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    sku: 1
}).skip(30).limit(10)
```

6.5 Quinta página

```
db.productos.find(
    {},
    {
        _id: 0,
```

```
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    sku: 1
}).skip(40).limit(10)
```

7. Paginación con filtros

También podemos paginar únicamente un subconjunto de productos.

7.1 Primera página de productos de Computo

```
db.productos.find(
    { categoria: "Computo" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    precio: -1
}).skip(0).limit(5)
```

7.2 Segunda página de productos de Computo

```
db.productos.find(
    { categoria: "Computo" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1
    }
).sort({
    precio: -1
}).skip(5).limit(5)
```

8. Combinar filtro, proyección, ordenamiento y límite

8.1 Productos con precio menor a $2,000 ordenados del más caro al más económico

```
db.productos.find(
    { precio: { $lt: 2000 } },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        categoria: 1,
        precio: 1
    }
).sort({
    precio: -1
}).limit(10)
```

8.2 Productos con stock mayor a 30 ordenados por stock

```
db.productos.find(
    { stock: { $gt: 30 } },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        stock: 1
    }
).sort({
    stock: -1
}).limit(10)
```

8.3 Productos con etiqueta `gaming` ordenados por precio

```
db.productos.find(
    { etiquetas: "gaming" },
    {
        _id: 0,
        sku: 1,
        nombre: 1,
        precio: 1,
        stock: 1
    }
).sort({
    precio: -1
})
```

Ejercicios de la Práctica 4

Para estos ejercicios se requiere la captura de pantalla de los resultados obtenidos debajo de cada ejercicio, guárdala en PDF y nómbralo: Practica04-Proyeccion-Ordenamiento-Paginacion.js

1. Mostrar únicamente sku, nombre y precio de todos los productos.
2. Mostrar sku, nombre, categoria y stock sin mostrar _id.
3. Consultar los productos de Audio mostrando solo nombre y precio.
4. Mostrar todos los campos excepto etiquetas.
5. Ordenar todos los productos por precio ascendente.
6. Ordenarlos por precio descendente.
7. Ordenar los productos por stock ascendente.
8. Ordenar los productos alfabéticamente por nombre.
9. Ordenar primero por categoría y posteriormente por precio descendente.
10. Mostrar los 5 productos más caros.
11. Mostrar los 5 productos más económicos.
12. Mostrar los 10 productos con menor stock.
13. Mostrar los 5 productos con mayor stock.
14. Mostrar los 3 productos de Audio más caros.
15. Mostrar los 5 accesorios más económicos.
16. Obtener la primera página de 10 productos ordenados por SKU.
17. Obtener la segunda página de 10 productos.
18. Obtener la cuarta página de 10 productos.
19. Mostrar una página de 5 productos de Computo ordenados por precio descendente.
20. Diseñar una consulta propia que combine filtro + proyección + ordenamiento + limitación y explicar qué información comercial obtiene.

Guardar el script al terminar la práctica

- Dentro de `mongosh`, escribe:

```
history()
```

Esto muestra el historial de comandos ejecutados durante la sesión.

- Después selecciona y copia el historial y pégalo en Visual Studio Code o Bloc de notas.
- Lo guárdalo como:

```
Practica04-Proyeccion-Ordenamiento-Paginacion.js
```

- Súbe a tu repositorio:
    - El archivo: Practica04-Proyeccion-Ordenamiento-Paginacion.js
    - El archivo: Practica04-Proyeccion-Ordenamiento-Paginacion.pdf (Con la portada)